

Regex – Pattern Matching



Wstęp

Istnieją trzy oddzielne podejścia do dopasowywania wzorców zapewniane przez PostgreSQL:

- tradycyjny operator SQL LIKE,
- nowszy operator SIMILAR TO (dodany w SQL:1999)
- oraz wyrażenia regularne w stylu POSIX.

Oprócz podstawowych operatorów "czy ten ciąg pasuje do tego wzorca?", dostępne są funkcje do wyodrębniania lub zastępowania pasujących podciągów oraz do dzielenia ciągu w pasujących miejscach.

ILIKE – case
insensitive

	A-Z last_name ▼
1	King
2	Khoo
3	Kaufling
4	King
5	Kumar

```
select      last_name
from employees
where      last_name ilike 'k%'
```

LIKE

	AZ last_name
1	King
2	Khoo
3	Kaufling
4	King
5	Kumar

```
select
    last_name
from employees
where
    last_name ~~ 'K%'
```

	AZ last_name
1	Yang
2	Garcia
3	James
4	Miller
5	Williams

```
select
    last_name
from employees
where
    last_name !~~ 'K%'
```

Szukanie liter

```
--szukanie ciągów z literą "k" case-sensitive
```

```
select
    last_name
from employees
where
    last_name ~ 'k'
```

```
--szukanie ciągów z literą "k" case-insensitive
```

```
select
    last_name
from employees
where
    last_name ~* 'K'
```

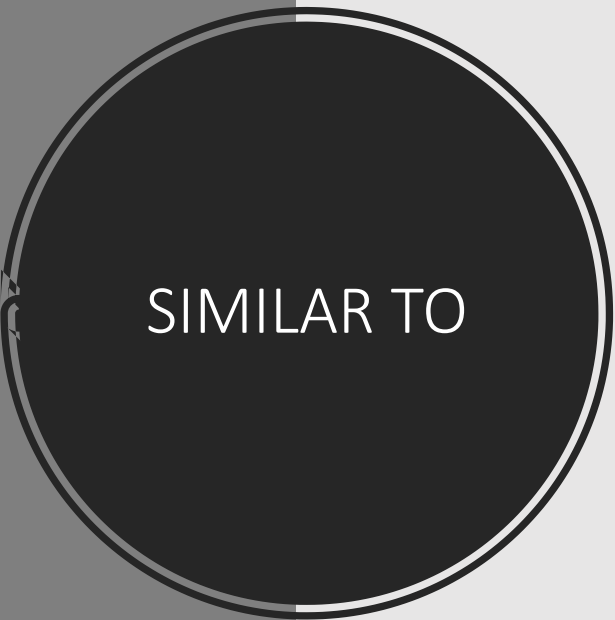
Szukanie liter

```
--szukanie ciągów bez literą "k" case-sensitive
```

```
select  
last_name  
from employees  
where  
last_name !~'k'
```

```
--szukanie litery "k" case-insensitive
```

```
select  
last_name  
from employees  
where  
last_name !~*'K'
```

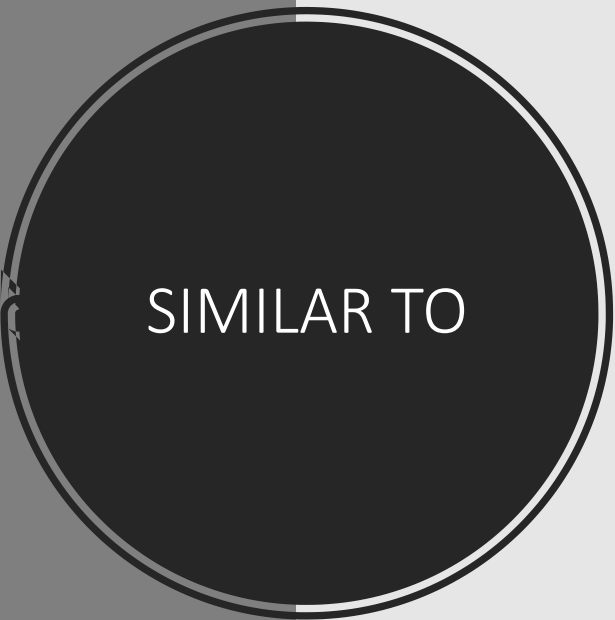


SIMILAR TO

Operator SIMILAR TO zwraca wartość true lub false w zależności od tego, czy jego wzorzec pasuje do podanego ciągu znaków.

Jest on podobny do LIKE, z wyjątkiem tego, że interpretuje wzorzec przy użyciu definicji wyrażenia regularnego w standardzie SQL.

Wyrażenia regularne SQL są ciekawym skrzyżowaniem notacji LIKE i zwykłej (POSIX) notacji wyrażeń regularnych.



SIMILAR TO

Operator SIMILAR TO w PostgreSQL jest mniej optymalny wydajnościowo od LIKE – operator LIKE jest szybszy i prostszy w przetwarzaniu.

Porównanie wydajności

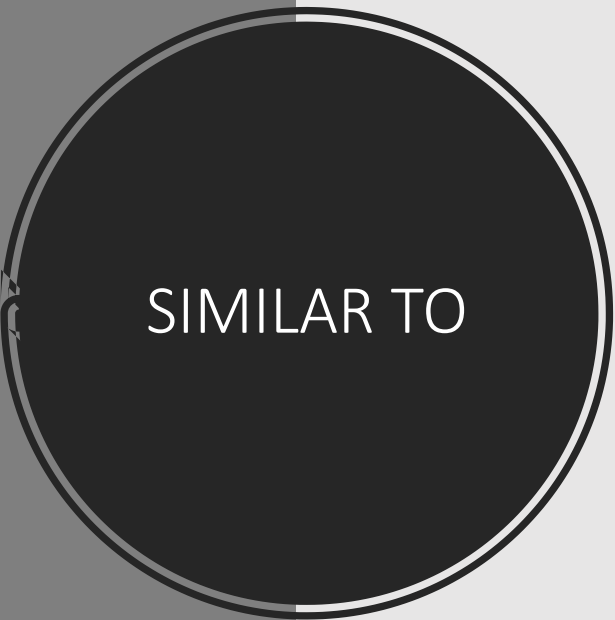
- **LIKE** – działa bardzo szybko, bo realizuje nieskomplikowane dopasowania wzorca i w wielu przypadkach może być wspierany przez indeksy (np. gdy wzorzec nie zaczyna się od %), dzięki czemu zapytania są bardzo wydajne.
- **SIMILAR TO** – obsługuje bardziej złożoną składnię, opartą na prostych wyrażeniach regularnych SQL, co powoduje większy narzut obliczeniowy i wolniejsze wykonanie niż LIKE w tych samych przypadkach.

Praktyczne wskazówki

Dokumentacja oraz benchmarki zalecają używanie LIKE do prostych dopasowań wzorców, gdyż jest to rozwiązanie szybsze.

SIMILAR TO warto wykorzystywać tylko wtedy, gdy konieczna jest większa elastyczność niż oferuje LIKE, np. do wzorców wymagających alternatyw (OR, przedziały znaków).

W skrócie: LIKE jest bardziej optymalny niż SIMILAR TO, gdy tylko można z niego skorzystać.



SIMILAR TO

Oprócz funkcji zapożyczonych z LIKE, SIMILAR TO obsługuje metaznaki dopasowywania wzorców zapożyczone z wyrażeń regularnych POSIX:

- | - oznacza naprzemiennność (jedną z dwóch alternatyw).
- * - oznacza powtórzenie poprzedniego elementu zero lub więcej razy.
- + - oznacza powtórzenie poprzedniego elementu jeden lub więcej razy.
- ? - oznacza powtórzenie poprzedniego elementu zero lub jeden raz.
- {m} - oznacza powtórzenie poprzedniego elementu dokładnie m razy.
- {m,} - oznacza powtórzenie poprzedniego elementu m lub więcej razy.
- {m,n} - oznacza powtórzenie poprzedniego elementu co najmniej m i nie więcej niż n razy.
- Nawiasy () - mogą być używane do grupowania elementów w pojedynczy element logiczny.
- Wyrażenie nawiasowe [...] - określa klasę znaków, podobnie jak w wyrażeniach regularnych POSIX.

Podobnie jak w przypadku LIKE, odwrotny ukośnik (\) wyłącza specjalne znaczenie któregośkolwiek z tych metaznaków.

SIMILAR TO

'abc'	SIMILAR TO	'abc'	true
'abc'	SIMILAR TO	'a'	false
'abc'	SIMILAR TO	'%(b d) %'	true
'abc'	SIMILAR TO	'(b c) %'	false
'-abc-'	SIMILAR TO	'%\mabc\M%'	true
'xabcy'	SIMILAR TO	'%\mabc\M%'	false

SIMILAR TO - substring

	A-Z last_name	A-Z match
13	Urman	[NULL]
14	Popp	[NULL]
15	Li	[NULL]
16	Khoo	Khoo
17	Baida	[NULL]
18	Tobias	[NULL]
19	Himuro	[NULL]

```
SELECT
last_name,
substring(last_name similar '%o{2}' escape '#') AS match
FROM employees
```

	A-Z ?column?	A-Z match
1	ab%%as	ab%%as

```
SELECT
'ab%%as',
substring('ab%%as' similar '%#{2}%' escape '#') AS
match
```

- Użycie ESCAPE definiuje znak, który zmienia znaczenie następnego znaku we wzorcu – „ucieka” jego specjalną funkcję.
- W przypadku SIMILAR TO i funkcji substring ... similar, pozwala dosłownie dopasowywać takie znaki jak: %, _, " oraz sam znak ucieczki.



POSIX

POSIX (Portable Operating System Interface) - specyfikacja wyrażeń regularnych

Główne elementy POSIX regex w PostgreSQL:

- Znaki specjalne:
 - . — dowolny znak
 - ^ i \$ — początek/koniec łańcucha
 - [abc] — klasa znaków
 - [^abc] — negowana klasa znaków
 - (...) — grupowanie
 - | — alternatywa (OR)
 - *, +, ? — kwantyfikatory
- Klasy znaków POSIX (zapis w [:nazwa:] wewnątrz []):
 - [:alpha:] — litery
 - [:digit:] — cyfry
 - [:alnum:] — litery lub cyfry
 - [:space:] — białe znaki (spacja, tab, nowa linia itd.)
 - [:punct:] — znaki interpunkcyjne
 - [:upper:], [:lower:], [:xdigit:] itd.
- Np. ^[:digit:]+\$ dopasuje string złożony wyłącznie z cyfr.

COUNT

regexp_count(string, pattern [, start [, flags]])

Parametry:

- string – tekst, w którym szukasz dopasowań.
- pattern – wyrażenie regularne POSIX
- start (opcjonalny) – od którego znaku tekstu zacząć liczenie (domyślnie 1)
- flags (opcjonalny) – kontroluje zachowanie regexu, np.:
 - 'i' → ignoruj wielkość liter (case-insensitive)
 - 'n' → kropka . dopasowuje także znak nowej linii
 - 'c' → wrażliwe na wielkość liter (domyślne)

```
SELECT regexp_count('ala ma kota, a kot ma ale', 'ma');
```

	123 regexp_count	
1		2

```
SELECT regexp_count('a1b2c3d4', '[:digit:]');
```

	123 regexp_count	
1		4

MATCH MATCHES

Regex_match – jedno dopasowanie - zwraca **pierwsze trafienie** zgodne z regexem

```
SELECT regexp_match('bananabanana', 'na');
```

	regexp_match
1	na

Regex_matches – wszystkie dopasowania- potrzebny jest flaga g (global), inaczej tylko jedno trafienie zwróci.

```
SELECT regexp_matches('bananabanana', '(na)', 'g');
```

	regexp_matches
1	[1]
1.1	na
2	[1]
2.1	na
3	[1]
3.1	na
4	[1]
4.1	na

COUNT MATCH MATCHES

Funkcja	Zwraca	Liczba dopasowań	Typ wyniku	Przykład	Wynik
regexp_match	Pierwsze dopasowanie	1	ARRAY	<code>SELECT regexp_match('ala ma kota', 'ma');</code>	<code>{"ma"}</code>
regexp_matches	Wszystkie dopasowania	wiele	SET OF ARRAY	<code>SELECT regexp_matches('ala ma kota, a kot ma ale', 'ma', 'g');</code>	<code>{"ma"} {"ma"}</code>
regexp_count	Liczba dopasowań	zlicza	INTEGER	<code>SELECT regexp_count('ala ma kota, a kot ma ale', 'ma');</code>	2



INSTR

regex_instr(string, pattern [, start [, occurrence [, return_opt [, flags]]]])

Zwraca pozycję (indeks znaku) w tekście, gdzie dane wyrażenie regularne zostało znalezione. Pozwala też określić które wystąpienie liczymy oraz czy zwracać początek czy koniec dopasowania.

Parametry:

- string – tekst wejściowy.
- pattern – wyrażenie regularne POSIX.
- start (opcjonalny) – od którego znaku zacząć szukanie (domyślnie 1).
- occurrence (opcjonalny) – które dopasowanie zwrócić (np. 2 = drugie wystąpienie).
- return_opt (opcjonalny):
 - 0 → pozycja początku dopasowania (domyślnie),
 - 1 → pozycja pierwszego znaku ZA dopasowaniem (koniec).
- flags (opcjonalny) – dodatkowe opcje regexu:
 - 'i' → ignoruj wielkość liter,
 - 'c' → uwzględniaj wielkość liter,
 - 'n' → . dopasowuje także newline.

INSTR

```
SELECT regexp_instr('bananabanana', 'na');
```

	123 regexp_instr
1	3

(bo pierwsze "na" zaczyna się na 3-cim znaku)

```
SELECT regexp_instr('bananabanana', 'na', 1, 2);
```

	123 regexp_instr
1	5

(drugie "na" zaczyna się na 5-tej pozycji)

```
SELECT regexp_instr('bananabanana', 'na', 1, 1, 1);
```

	123 regexp_instr
1	5

(bo pierwsze "na" kończy się na 4, a zwrócona jest pozycja po końcu = 5)

```
SELECT regexp_instr('PostgresREGEX', 'regex', 1, 1, 0, 'i');
```

	123 regexp_instr
1	9

(dopasowanie zaczyna się przy "R" na 9-tej pozycji)



REPLACE

regex_replace(string, pattern, replacement [, flags])

Pozwala **zamieniać fragmenty tekstu pasujące do wzorca**.

Parametry:

- string → tekst źródłowy, w którym ma nastąpić zamiana.
- pattern → wyrażenie regularne POSIX, które wskazuje fragmenty do podmiany.
- replacement → tekst, który wstawi się w miejsce dopasowania.
 - można używać odwołań do grup: \1, \2 itd.
- flags (opcjonalne) – zachowanie funkcji:
 - 'g' → global: zamień wszystkie dopasowania (domyślnie tylko 1. wystąpienie)
 - 'i' → ignoruj wielkość liter
 - 'n' → . dopasowuje także znaki nowej linii
 - 'c' → dopasowanie z uwzględnieniem wielkości liter (domyślnie)

REPLACE

```
SELECT regexp_replace('ala ma kota, a kot ma ale', 'ma', 'MIAU');
```

	A-Z regexp_replace
1	ala MIAU kota, a kot ma ale

(Tylko **pierwsze** "ma" zostało podmienione)

```
SELECT regexp_replace('ala ma kota, a kot ma ale', 'ma', 'MIAU', 'g');
```

	A-Z regexp_replace
1	ala MIAU kota, a kot MIAU ale

(Wszystkie wystąpienia "ma" zamienione)

```
SELECT regexp_replace('Jan Kowalski', '([A-Za-z]+) ([A-Za-z]+)', '\2, \1');
```

	A-Z regexp_replace
1	Kowalski, Jan

(Zamiana miejscami: \1 = pierwsza grupa, \2 = druga grupa)

```
SELECT regexp_replace('PostgresREGEX', 'regex', 'match', 'ig');
```

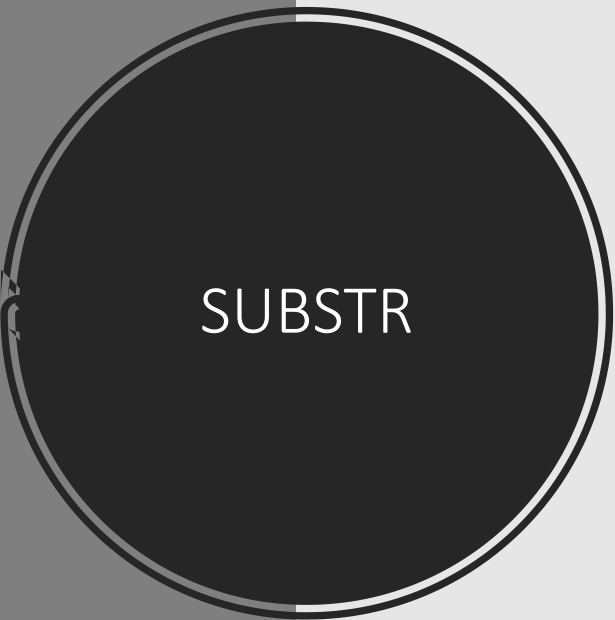
	A-Z regexp_replace
1	Postgresmatch

(regex dopasowało się niezależnie od wielkości liter, i zamieniono wszystkie wystąpienia)

```
SELECT regexp_replace('Tel: +48-123-456-789', '[^0-9]', '', 'g');
```

	A-Z regexp_replace
1	48123456789

(Zostawiono tylko cyfry)



SUBSTR

regexp_substr(string, pattern [, start [, occurrence [, flags]]])

Pozwala **zamieniać fragmenty tekstu pasujące do wzorca.**

Parametry:

- string – przeszukiwany tekst.
- pattern – regex (POSIX).
- start (opc.) – od którego znaku zacząć dopasowanie (domyślnie 1).
- occurrence (opc.) – które wystąpienie zwrócić (1 = pierwsze, 2 = drugie...).
- flags (opc.) – kontrola regexu:
 - 'i' → ignoruj wielkość liter
 - 'g' → w tym przypadku raczej nie używane (bo zwraca się tylko jedno wystąpienie)
 - 'n' → kropka . dopasowuje też znak nowej linii

SUBSTR

```
SELECT regexp_substr('ala ma kota', 'ma');
```

	A-Z regexp_substr
1	ma

(Zwraca pierwsze dopasowanie)

```
SELECT regexp_substr('bananabanana', 'na', 1, 2);
```

	A-Z regexp_substr
1	na

(Drugie wystąpienie na)

```
SELECT regexp_substr('a1b2c3d4', '[:digit:]', 1, 3);
```

	A-Z regexp_substr
1	3

(Trzecia cyfra w ciągu)

```
SELECT regexp_substr('PostgresREGEX', 'regex', 1, 1, 'i');
```

	A-Z regexp_substr
1	REGEX

(Dopasowanie znalezione bez względu na wielkość liter)

```
SELECT regexp_substr('Ala ma kota i psa', '\w+', 1, 4);
```

	A-Z regexp_substr
1	i

(bo \w+ = słowo, a czwarte słowo tu to "i")